Session 07 — Dictionaries

Python for Everyone · Lecture notes

> **How to read the code boxes:** grey boxes with a yellow bar contain Python code. The box labelled **Output** next to it (or below it) shows what Python prints when you run that code. In a Jupyter notebook, if the last line of a cell is just a value (like `age_dict` below), the notebook displays it; this is also shown as output.

Dictionaries

- In a **list**, values are looked up by their **position** (index, counting from 0): `L[1]`.
- In a **dictionary**, values are looked up by a **key**, like looking up the definition (value) of a word (key) in a real dictionary.
- A dictionary is written in curly brackets `{}` as `key: value` pairs, separated by commas.

```
age_dict = {"Marton": 40, "Vera": 5, "Ilona": 1}
age_dict["Marton"]
```

Output:

```
40
```

- Looking up a key that is not in the dictionary gives a `KeyError`:

```
age_dict["Zsofia"]
```

Output:

```
KeyError: 'Zsofia'
```

- Assigning to a key **changes** its value if the key exists, and **adds** a new entry if it does not:

```
age_dict["Ilona"] = 2      # change
age_dict["Zsofia"] = 38    # add
age_dict
```

Output:

```
{'Marton': 40, 'Vera': 5, 'Ilona': 2, 'Zsofia': 38}
```

- `{}` is an empty dictionary; you can fill it up entry by entry.

Mutable and immutable

- Lists and dictionaries can be changed: they are **mutable**. Strings cannot be changed: they are **immutable**.

```
L = [1, 2, 3]
L[1] = "wow!"
L
```

Output:

```
[1, 'wow!', 3]
```

```
s = "told"
s[0] = "s"
```

Output:

```
TypeError: 'str' object does not support item assignment
```

Rules for keys and values

- Keys are **unique**: a key can appear only once.
- Keys must be **immutable**: strings and numbers are fine, lists are not.

- Values can repeat and can be anything.

```
D = {}
D["one"] = 1
D[1] = 1
D
```

Output:

```
{'one': 1, 1: 1}
```

```
D[["one", 1]] = 1
```

Output:

```
TypeError: unhashable type: 'list'
```

Things you can do with a dictionary

- `len(d)` gives the number of entries (key: value pairs): `len(age_dict)` is `3` for `{"Marton": 40, "Vera": 5, "Ilona": 2}`.
- `key in d` tests whether the dictionary contains a key. Use it to avoid a `KeyError`:

```
age_dict = {"Marton": 40, "Vera": 5, "Ilona": 2}
key = "Marton"
if key in age_dict:
    print(f"The age of {key} is {age_dict[key]}")
else:
    print("Age unknown.")
```

Output:

```
The age of Marton is 40
```

Looping over a dictionary

- A `for` loop over a dictionary goes through its **keys**. `for key in age_dict.keys():` does the same, more explicitly.

```
for key in age_dict:
    print(key)
```

Output:

```
Marton
Vera
Ilona
```

- `.values()` goes through the values:

```
for value in age_dict.values():
    print(value)
```

Output:

```
40
5
2
```

- `.items()` gives the key and the value together. This `for` loop has **two** variables:

```
for key, value in age_dict.items():
    print(key, value)
```

Output:

```
Marton 40
Vera 5
Ilona 2
```

- The names `key` and `value` are your choice; pick names that make the code easy to read (e.g. `for title, rating in movie_ratings.items():`).

Building dictionaries

- Just like a list (start with `[]`, then `append`), a dictionary can be built in a loop: start with `{}`, then add entries.

```
words = ["Wilde", "mochas", "arcs", "simpers", "dynamically", "Chesterfield",
         "swilling", "reaped", "jeering", "haler", "accessioned", "Rodger"]
lengths_dict = {}
for w in words:
    lengths_dict[w] = len(w)
lengths_dict
```

Output:

```
{'Wilde': 5, 'mochas': 6, 'arcs': 4, 'simpers': 7, 'dynamically': 11, 'Chesterfield': 12,
'swilling': 8, 'reaped': 6, 'jeering': 7, 'haler': 5, 'accessioned': 11, 'Rodger': 6}
```

Structured data

- Dictionaries are great for storing **records**, e.g. the title, director and rating of a movie. In a list you must remember what each position means (`movie_list[1]` is the director?); in a dictionary the key says it.

```
movie_list = ['Repo Man', 'Alex Cox', 10]
movie_dict = {
    'title': 'Repo Man',
    'director': 'Alex Cox',
    'rating': 10
}
movie_dict['director']
```

Output:

```
'Alex Cox'
```

- Many records: use a **list of dictionaries**.

```
movies = [
    {'title': 'Titanic', 'director': 'James Cameron', 'rating': 4},
    {'title': 'Werckmeister Harmonies', 'director': 'Bela Tarr', 'rating': 9},
    {'title': 'Repo Man', 'director': 'Alex Cox', 'rating': 10},
]
print(movies[0]['title'])
for m in movies:
    print(m['title'])
```

Output:

```
Titanic
Titanic
Werckmeister Harmonies
Repo Man
```

- Combining lists and dictionaries like this is a very common, self-explanatory way to store data.

Why dictionaries?

- You could imitate a dictionary with two lists, one for the keys and one for the values. But to look up a key you must check the keys one by one: with N entries this takes up to N steps.

```
keys   = ['apple', 'banana', 'persimmon']
values = [4, 3, 5]
key = 'persimmon'
for i in range(len(keys)):
    if keys[i] == key:
        print(values[i])
```

Output:

```
5
```

- A dictionary finds a value **in a single step**, no matter how large it is.
- **General lesson:** choosing the right way to store your data makes your code both easier to write and faster to run.

Common mistakes

- **Keys must match exactly:** `age_dict["marton"]` gives `KeyError: 'marton'` because keys are case sensitive.
- **Forgetting the quotes:** `age_dict[Marton]` looks for a *variable* called `Marton`: `NameError: name 'Marton' is not defined`.
- **Using a position:** `age_dict[0]` does not give the first entry; it looks for the key `0`: `KeyError: 0`.
- **`in` checks keys, not values:** `40 in age_dict` is `False`, while `"Vera" in age_dict` is `True`.
- **Two loop variables without `.items()`:** `for key, value in age_dict:` gives `ValueError: too many values to unpack (expected 2)`.
- **A list as a key:** gives `TypeError: unhashable type: 'list'`.

Summary

Term / code	Meaning
dictionary	collection of `key: value` pairs
`{"a": 1, "b": 2}`	create a dictionary; `{}` is an empty one
`d[key]`	look up the value of a key
`d[key] = value`	change the value of a key, or add a new entry
`key in d`	`True` if the dictionary contains the key
`len(d)`	number of entries
`for key in d:`	loop over the keys (same as `d.keys()`)
`for value in d.values():`	loop over the values
`for key, value in d.items():`	loop over keys and values together
mutable / immutable	can / cannot be changed (list, dict / string)
list of dictionaries	a common way to store records (structured data)
`KeyError`	the key is not in the dictionary
`TypeError: unhashable type`	this type cannot be used as a key (e.g. a list)